Epistemos — Doctrine for Agentic Runtime, Local Model Autonomy, and Generative UI

Scope: Implementation doctrine for a Swift 6 + Rust (UniFFI) + Metal + MLX-Swift + GRDB macOS PKM app targeting M2 Pro. Current as of April 2026.

Orientation: You are building a *host* app that must (a) piggyback on whatever AI tooling the user has already installed, (b) keep working fully offline against Qwen3-class local models, and (c) let those models generate durable, native-feeling UI. Every decision below is ordered to protect three invariants: notarization survival, offline-first correctness, and a single on/off toggle UX.

PART 1 — RUNTIME ARCHITECTURE: FOUR PATHS, ONE TOGGLE

1.0 The four paths, ranked

Path	What user installs	Auth inherited	Cost model	Notarization risk	Sandbox feasibility	Recommended role
A. Subprocess CLIs	`claude`, `codex`, `gemini` (npm/Homebrew)	Yes (OAuth/keychain on host)	User's Pro/Max or their API key	Developer ID: low. MAS: **disqualifying** if sandboxed	Must ship **Developer ID-notarized, non-sandboxed** variant	"Power mode" default when CLIs detected
B. Official SDKs / API keys	Nothing; user pastes API key	No (Epistemos stores keys)	User API billing, you absorb nothing	Clean	Fully sandbox-compatible	Default for clean installs
C. MCP (server + client)	An MCP-capable client (for server role) or nothing (for client role)	N/A (tool-level only)	Whatever drives the loop pays	Clean	Sandbox-compatible for network MCP; stdio MCP requires sandbox exemption	Universal tool surface, dual role
D. Hybrid adaptive	Detected at runtime	Combined	Route per-task	Depends on chosen leaf	Two build variants	**Shipping configuration**

The takeaway, which determines everything else in this document: **you cannot ship one binary that both spawns `claude` / `codex` / `gemini` CLIs and passes Mac App Store review as a sandboxed app.** Apple's rule is unambiguous: any executable you spawn must be signed with `com.apple.security.app-sandbox` and `com.apple.security.inherit`, Apple Developer which Node-based CLIs installed via npm are not (Apple Developer Forums). Even the official MCP Swift SDK example explicitly warns that stdio MCP servers require disabling sandboxing Medium (gsabran/mcp-swift-sdk), a constraint echoed by every Swift MCP wrapper in the wild (jamesrochabrun). **Doctrine: ship Epistemos as a Developer ID–notarized app with Hardened Runtime but *not* App Sandbox.** If you ever want MAS distribution, split the subprocess-spawning piece into a separately-distributed "Epistemos Power Pack" helper or document that Power Mode requires the direct download. Do not fight this.

1.1 Path A — Subprocess spawning (the "Power Mode")

1.1.1 What's actually available (April 2026)

All three vendor CLIs now have mature, machine-parseable non-interactive modes with event streams:

- **Claude Code —** `claude -p "<prompt>" --output-format stream-json --verbose --include-partial-messages` emits newline-delimited JSON events Claude including `system/api_retry` events for rate limits, each carrying a `session_id` for replay/resume Background Claude (Anthropic docs, Background Claude). Schema-constrained output is supported via `--json-schema`. Claude The flag `--bare` strips auto-discovery (hooks, skills, plugins, MCP, CLAUDE.md, keychain OAuth) — this is the mode Anthropic explicitly recommends for SDK/scripted callers and states *will become the default for* `-p` Claude (Anthropic docs). `--allowedTools "Read,Edit,Bash"` whitelists, `--resume <session_id>` and `--continue` resume sessions. Claude
- **OpenAI Codex CLI —** `codex exec "<prompt>" --json` streams NDJSON events; Openai `codex exec resume --last` or `codex exec resume <SESSION_ID>` resumes. Openai `--output-schema ./schema.json` forces structured final output, Openai `-o ./file.json` writes final message to disk, Openai `--ephemeral` avoids persisted rollouts. Openai Auth is via `codex login` (ChatGPT OAuth) **or** `CODEX_API_KEY` (the only place the env var works is `codex exec`) Openai (OpenAI developers, CLI reference). Codex now also ships a `codex mcp` subcommand that runs Codex itself as an MCP server Openai — meaningful for path C.
- **Gemini CLI —** `gemini -p "<prompt>" --output-format json` returns a single envelope with full usage stats (per-model token accounting, tool call decisions); gemini-cli `stream-json` exists for incremental events GitHub (geminicli.com, google-gemini/gemini-cli). `--yolo` auto-approves tools (dangerous), `--non-interactive` forces scripted mode. Inventive HQ Auth via `GOOGLE_API_KEY` or `GOOGLE_APPLICATION_CREDENTIALS`. Inventive HQ

1.1.2 Authentication inheritance and the licensing trap

When you spawn `claude -p` from Epistemos, it by default reads `~/.claude/` (OAuth tokens, Pro/Max subscription). That means your user's *existing subscription* pays — which is exactly the UX the user described. But: Anthropic's Agent SDK ToS explicitly states *"Anthropic does not permit third-party developers to offer Claude.ai login or to route requests through Free, Pro, or Max plan credentials on behalf of their users"* The New Stack (Anthropic docs). The Anthropic PR team clarified in early 2026: *personal use is fine; if you are building a business on top of the Agent SDK, use an API key* The New Stack (The New Stack).

Doctrine: Frame subprocess CLI usage as *"Epistemos launches your existing Claude Code with a prompt you author"* — not *"Epistemos provides Claude access."* In the UI, never suggest Pro/Max is a substitute for paying Anthropic. In Path A, always spawn `claude -p` with `--bare` so Epistemos isn't accidentally picking up the user's keychain OAuth in a way that looks like rerouting — and add a per-project override for `ANTHROPIC_API_KEY` when the user provides one. OpenAI has not published equivalent restrictive language; Codex OAuth via ChatGPT account is explicitly supported for `codex exec` (OpenAI devs).

1.1.3 PTY vs pipes

Both Claude Code and Codex behave correctly under pipes when given `-p` / `exec` flags; they detect non-TTY and skip the Ink/Ratatui TUI. You do **not** need a PTY for Path A — stdin/stdout pipes are sufficient. PTY is only necessary if you want to embed the full interactive TUI inside a Terminal panel (out of scope for Epistemos). Gemini CLI had a documented `DEBUG` env var bug that caused it to hang waiting for a debugger in non-TTY mode; that was patched GitHub (gemini-cli PR #14580) — still, *always scrub* `DEBUG`, `NODE_OPTIONS`, `LD_PRELOAD`, *and reset* `PATH` *in the spawn environment*. The Claude Agent Rust SDK already does this kind of environment filtering by default Rust (claude_agent_sdk on docs.rs).

1.1.4 Discovery

Probe order for each CLI:

1. `which claude` / `which codex` / `which gemini` via `NSWorkspace` + `/usr/bin/env which` (never just `PATH`; many users have Homebrew in `/opt/homebrew/bin`, nvm at `~/.nvm/versions/node/*/bin`, pnpm globals, mise/asdf shims).
2. Known install paths: `/opt/homebrew/bin`, `/usr/local/bin`, `~/.npm-global/bin`, `~/.bun/bin`, `~/.volta/bin/<name>`, `~/.local/bin`.
3. Invoke `<cli> --version` and parse. Pin minimum versions (Claude Code ≥ 2.x, Codex ≥ 0.80, Gemini CLI latest

stable).

4. Cache result in GRDB with a 24h TTL; invalidate on app launch if `stat` mtime of the binary changed.

1.1.5 Failure modes

- CLI absent → offer one-click `brew install` or `npm i -g` (present via a shell-script panel; don't actually run `brew` from inside the app).
- CLI present, unauthenticated → surface the exact `claude login` / `codex login` / `gemini` command; do not attempt to inject tokens.
- CLI hangs → enforce a hard `tokio::time::timeout` around the child; on timeout, send `SIGTERM`, then `SIGKILL` after 2s.
- Stdout exceeds buffer (there's a known Codex regression that produced 1.8 GB outputs in early 2026 GitHub (openai/codex#9029)) → stream to a temp file, not memory; cap at 200 MB per turn.

1.1.6 Sandbox/entitlements — the hard truth

- You need `com.apple.security.cs.allow-unsigned-executable-memory` nowhere (that's JIT).
- You need Hardened Runtime *without* `com.apple.security.app-sandbox`.
- You need `com.apple.security.cs.allow-jit` only for MLX Metal compilation — verify with Apple's `otool` / `codesign -d --entitlements -` after build.
- You cannot use `com.apple.security.inherit` with an unsigned Node script. Claude/Codex/Gemini CLIs are all Node/JS shipped via npm, unsigned by npm, so `inherit` is moot.
- **Verdict:** Ship Epistemos as a Developer ID app with Hardened Runtime; notarize it; do **not** put `<key>com.apple.security.app-sandbox</key><true/>` in entitlements. This is how Cursor, Zed, Warp, Raycast, and every other "agentic IDE/launcher" on macOS ships.

1.2 Path B — Official SDKs/APIs

1.2.1 The Claude Agent SDK is the right abstraction — and it has a mature Rust binding

Anthropic rebranded "Claude Code SDK" → **Claude Agent SDK** in late 2025, explicitly scoping it beyond coding: *"The agent harness that powers Claude Code (the Claude Code SDK) can power many other types of agents, too"* Anthropic (Anthropic). The official SDKs are Python and TypeScript. There is **no first-party Swift SDK and no first-party Rust SDK**.

Third parties have filled the gap with two different topologies:

- **`claude-agent-sdk` on crates.io** (docs.rs) wraps the CLI via stdio JSON envelopes — essentially Path A wearing a Rust trench coat. It provides `query()`, `ClaudeSDKClient`, `SdkMcpServer`, hooks, and a `PermissionManager`. Rust It automatically filters dangerous env vars (`LD_PRELOAD`, `NODE_OPTIONS`, `PATH`) and surfaces `ClaudeError::CliNotFound`. Rust Same distribution constraint as Path A.
- **`claude-agent` on docs.rs** (docs.rs/claude-agent) is a pure-API implementation against the Anthropic Messages API *without CLI subprocess dependencies*. Rust This is what you want for Path B.

For OpenAI, Google, and mixed providers, the canonical Rust option is **Rig (`rig-core`)** from 0xPlaygrounds (rig.rs, GitHub). Rig provides `CompletionModel`, `EmbeddingModel`, `Agent`, and `AgentBuilder` with a consistent API across Anthropic, OpenAI, Gemini, Cohere, and ~15 others, full streaming, a `tool_macro` derive Rig on top of `schemars::JsonSchema`, dynamic tool retrieval via vector stores, and OpenTelemetry tracing hooks. Production users include VT Code, Dria, Neon's `app.build` v2, and Nethermind (crates.io). A companion crate `llm-coding-tools-rig` ships pre-built `ReadTool`, `GrepTool`, `GlobTool`, `BashTool`, and `TodoTools` that are directly compatible with Rig's `Agent` (docs.rs) — i.e. you get Claude Code's tool surface as drop-in Rust primitives.

Note on "yoagent": I could not find an actively maintained Rust crate named `yoagent` on crates.io/GitHub as of April 2026. Treat your prior reference as either a private fork or mis-remembered; use `rig` + `claude-agent` (pure API) + the official MCP Rust SDK instead. `mini-agent` (crates.io) is a minimal alternative with a cleaner `LlmProvider` / `Tool` trait surface crates.io if you want to own the loop.

1.2.2 Tool surface to replicate

To reach Claude Code parity purely through API + client-side tools, implement these tools on the Rust side (mirrors Anthropic's Claude Code toolset + Aider/Cursor patterns):

```
Read(path)  Edit(path, old, new)  Write(path, content)
MultiEdit(path, [edits])  Glob(pattern)  Grep(pattern, path)
Bash(cmd, timeout, cwd)  WebFetch(url)  WebSearch(query)
TaskCreate/Read/Update  NotebookEdit  Todo read/write
```

Route all of these through `uniffi::Object` exposed to Swift so the Swift layer can render tool-call cards. Claude Code's own tool names (Read/Edit/Bash) are the de facto standard and what every Rig tool library targets.

1.2.3 Cost model: user provides API key, you absorb zero. Offer per-provider budget caps enforced in Rust before each API call; surface usage via Anthropic's `usage` field, OpenAI's `usage.total_tokens`, Gemini's `stats.models.*.tokens.total` (shown in the headless JSON envelope).

1.3 Path C — MCP, in both roles

1.3.1 MCP is now the wire protocol for everything

As of December 2025, Anthropic donated MCP to the Agentic AI Foundation The New Stack under the Linux Foundation, with co-founding support from OpenAI, Block, Google, Microsoft, AWS, Cloudflare, and Bloomberg Anthropic (Anthropic). Concrete support status in April 2026:

- **Anthropic Claude Desktop + Claude Code**: native MCP client, both stdio and HTTP/SSE transports.
- **OpenAI**: MCP support across Agents SDK, Responses API, and ChatGPT since March 2025 Pento (The New Stack); Codex CLI has `codex mcp add/list/remove` and can itself be run as an MCP server via `codex mcp serve`.
- **Google**: Gemini models and Vertex support MCP; Google rolled out fully-managed remote MCP servers Google Cloud for BigQuery, Cloud Run, GKE, Maps, etc. Google Cloud (Google Cloud blog).
- **Microsoft**: Semantic Kernel, Azure OpenAI, Wikipedia Windows 11 all integrate.
- **Ecosystem**: 10,000+ public servers, Anthropic 97M monthly SDK downloads Pento (Pento, Anthropic). The latest spec revision is dated 2025-11-25 Imagine-works GitHub (Swift SDK README).

1.3.2 Epistemos should be *both* an MCP server and an MCP client

- **As a server** (exposing your vault/graph/tools): any user running Claude Desktop, Claude Code, ChatGPT Desktop, Cursor, Zed, VS Code, or Gemini CLI can drive Epistemos's agent loop using *their* model of choice. This is the "we don't need to ship an agent at all" escape valve and the honest answer to the user's "we want everything without them needing those clients separately" goal: for power users who already have those clients, MCP is the cleanest integration and doesn't require Epistemos to proxy anything.
- **As a client** (consuming external MCP servers): Epistemos's own local or cloud agent loop can pick up `filesystem`, `github`, `memory`, `playwright`, `time`, `fetch` servers users already have configured in `~/.claude/mcp.json` or `~/.codex/config.toml`. Slurp those configs at launch, reuse them.

1.3.3 Swift implementation

Use the **official Swift SDK for MCP** (modelcontextprotocol/swift-sdk), maintained in collaboration with Loopwork. MCP Server Finder MCP Server Finder It implements both client and server against the 2025-11-25 spec, ships `StdioTransport`, `HTTPServerTransport`, and a `withMethodHandler(CallTool.self)` pattern. Artem Novichkov For the pure-

Rust server half, use `rmcp` or equivalent (the official MCP Rust SDK). Prefer **HTTP/SSE or Streamable HTTP transport for the server role** — this is sandbox-friendly and auth-ready (OAuth 2.1, which became mandatory in March 2025's spec update). Medium Reserve stdio transport for the client role where you're consuming local-only servers.

1.3.4 MCP Apps (SEP-1865): the UI extension that changes things

On January 26, 2026, Anthropic and OpenAI jointly released **MCP Apps** (MCP Blog, The Register). This extension allows an MCP server to return *interactive UI components* (HTML rendered in sandboxed iframes, with pre-declared templates, auditable host messages, and user-consent flows) Inkeep that the client renders inline. Launch partners: Hallam Amplitude, Asana, Box, Canva, Clay, Figma, Hex, monday.com, Slack. The Register This is directly relevant to Area 3 — it is the industry answer to generative UI within chat — and it's what you should design your internal schema to be *compatible with* rather than reinvent. See §3.

1.4 Path D — Hybrid adaptive (the shipping configuration)

1.4.1 Detection + routing layer

```
Epistemos launch
  └─ ProviderDiscovery (Rust)
     ├─ probe `claude`, `codex`, `gemini` binaries + versions
     ├─ probe ANTHROPIC_API_KEY, OPENAI_API_KEY, GOOGLE_API_KEY env + keychain
     ├─ probe MLX model cache at ~/.cache/epistemos/mlx/
     └─ emit ProviderMatrix { cli_available, api_keys, local_models, mcp_servers }
```

Then a **Router** makes per-task decisions:

```
enum Provider {
    LocalQwen3_4B,          // fast, free, unreliable beyond ~3 hops
    LocalQwen3_9B,          // planner/router, 256K ctx
    ClaudeAgentSDK_API,     // path B
    ClaudeCodeCLI,          // path A
    CodexCLI,               // path A
    GeminiCLI,              // path A
    OpenAIResponses,        // path B
    GeminiAPI,              // path B
    MCPOnly,                // path C: no model, pure tool proxy
}

trait TaskClassifier {
    fn classify(&self, task: &Task) -> TaskKind;
    // kind ∈ { Chitchat, NoteWrite, Search, SimpleEdit,
    //          CodeEdit, MultiStepAgent, LongHorizonAgent }
}

trait Policy {
    fn pick(&self, matrix: &ProviderMatrix, kind: TaskKind, budget: Budget)
        -> Vec<Provider>; // ordered preference list
}
```

1.4.2 Default policy (documented, user-editable):

Task kind	Preferred	Fallbacks
Chitchat / quick note	Qwen3-4B local	Haiku API → Gemini Flash API
Note write / rewrite	Qwen3-4B local	Sonnet API
Vault search / graph walk	Qwen3-4B + local tools	n/a (always local)
Single-file edit	Claude Code CLI (if installed)	Claude Agent SDK (API) → Codex API

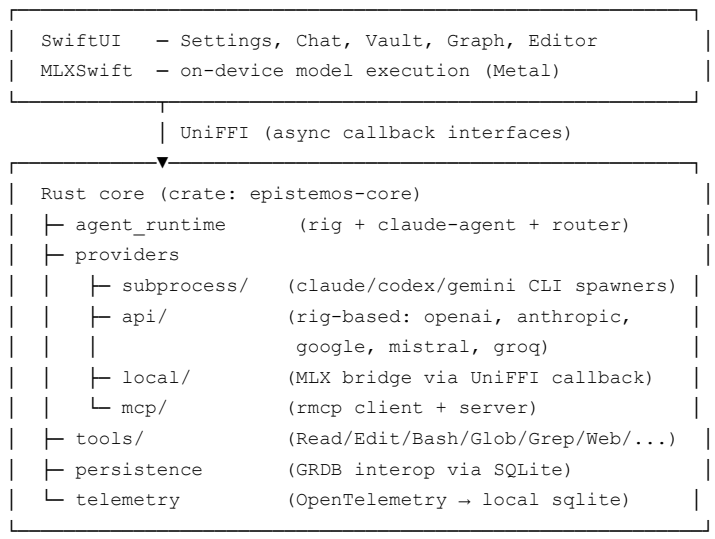
Multi-step agent (<10 hops)	Claude Code CLI	Claude Agent SDK (API) → Codex CLI
Long-horizon agent	Claude Agent SDK (API) with checkpointing	Codex CLI `--resume`
"This is too complex" escalation	Claude Opus class	GPT-5.4 → Gemini 3 Pro

1.4.3 UX for the toggle: Single Settings pane with three radio modes:

1. **Auto** (default) — routes per table above, shows the chosen provider inline on each message ("routed to: Claude Code CLI · Pro subscription").
2. **Local only** — Qwen3 4B/9B, never calls network. Escalation chain disabled.
3. **Manual** — user picks per-chat or per-task from a provider palette; routing is explicit.

Plus a secondary **Advanced** sheet with per-task overrides (a table mirroring the matrix above), global budget caps, and "allow subprocess CLIs: yes/no" (for users on corp Macs where they don't want Epistemos touching child processes).

1.5 Layering onto Swift + Rust + UniFFI



Critical UniFFI shape: expose an `AgentSession` object with async methods `send_message`, `cancel`, `approve_tool`, `deny_tool`, and a **callback interface** `AgentEventSink` that Swift implements with `onAssistantDelta`, `onToolCallRequested`, `onToolCallResult`, `onRouterDecision`, `onCostAccrued`, `onSessionCheckpoint`. UniFFI supports async and callback interfaces in 0.28+; this is the right boundary because it lets Rust own the loop and Swift render.

For **MLX bridge**: Rust cannot call MLX-Swift directly. Invert the dependency — Rust's `local` provider is a trait whose implementation is a Swift object bridged back via a UniFFI callback interface `LocalLLMProvider` with `generate_stream(prompt, tools) -> AsyncSequence<Token>`. Swift owns the MLX session; Rust only marshals. This is exactly how the Claude Agent Rust SDK wraps TypeScript (srothgan/claude-code-rust) and the standard pattern for MLX-Swift integration.

PART 2 — MAKING QWEN3-4B AGENTIC (WITH ESCAPE HATCHES)

2.0 Ground truth on Qwen3-4B

Qwen3-4B (both `Qwen/Qwen3-4B-MLX-4bit` and `-8bit`) ships with native tool-calling trained in, supports thinking/non-thinking mode toggles, 32K context natively extensible to 131K via YaRN, and is explicitly marketed for "precise integration with external tools … achieving leading performance among open-source models in complex agent-based

tasks" (HuggingFace). It runs on MLX with `mlx_lm ≥ 0.25.2`. Qwen-Agent is the official framework and it natively consumes MCP server configs (HF). The newer Qwen3.6-35B-A3B (MoE, 3B active) is a legitimate "9B-class router" candidate given its MLX availability and repo-level agentic training (unsloth/Qwen3.6-35B-A3B-UD-MLX-4bit), with 262K native context.

On an M2 Pro (16GB minimum, 32GB recommended): Qwen3-4B-MLX-4bit runs at ~30–50 tok/s decode, ~200–400 tok/s prefill. 8-bit is 15–25 tok/s decode. Qwen3.6-35B-A3B-4bit on 32GB is viable at ~8–15 tok/s because only 3B parameters are active per token.

2.1 Tier 1 — Tool use only

Prompt shape: Qwen-Agent's tool calling uses Hermes/ChatML-style tool tags. Don't reinvent; reuse the `Qwen/Qwen3-4B-MLX-4bit` tokenizer's built-in chat template, which already emits `<tool_call>...</tool_call>` blocks. On the Rust side, the `rig::tool::Tool` trait + `schemars::JsonSchema` derive (Rig docs) maps 1:1 — define tools once in Rust, expose them both to Rig-driven cloud providers *and* to the MLX local path via Swift↔Rust callback.

Structured outputs: MLX-Swift doesn't yet have a first-class equivalent of llama.cpp's GBNF grammars or Outlines. Three workable options on M2 Pro, in order of reliability:

1. **Two-step verification** — let Qwen emit JSON, validate with `serde_json::from_str::<T>`; on failure, re-prompt with the parse error appended. Qwen3-4B succeeds first-shot on simple schemas >90% of the time in practice.
2. **Logit processors via `mlx-swift-examples`** — MLX-LM Swift exposes a `logitProcessors` hook; port a JSON-mode biaser (ban non-JSON start tokens after whitespace). Nontrivial but bounded work.
3. **Apple Foundation Models `@Generable` / `@Guide`** (createwithswift.com) — if you're willing to use Apple's on-device LLM *in parallel* to Qwen for schema-constrained generation of UI JSON (which is what you actually need in §3), this is the clean answer on macOS 26+. `@Generable struct` auto-generates a schema the model will emit.

Latency on M2 Pro: a full tool-use turn (prompt 2K tokens + 200 token response + JSON parse) lands at ~1.5–3s. Acceptable for UI, not for typing autocomplete.

2.2 Tier 2 — Structured agentic loops

Architecture: ReAct in Rust, *not* in the model. The model is a stateless function `prompt → tool_call_or_answer`; the Rust loop owns state, step counter, tool execution, budget, and termination. Template:

```
let mut state = AgentState::new(task, max_steps=12, budget=USD(0.10));
loop {
    let step = model.step(&state.render_prompt()).await?;    // one LLM call
    state.record(step.thought, step.action);
    if let Action::Finish(answer) = step.action { break answer; }
    let observation = tools.execute(step.action, &policy).await?;  // sandboxed
    state.record_observation(observation);
    if state.step >= state.max_steps { return escalate(&state); }
    if state.spent >= state.budget   { return escalate(&state); }
}
```

Key guardrails:

- **Planner/executor split** — first turn is `Qwen3.6-35B-A3B` ("router") generating a plan as a JSON task list; subsequent turns are `Qwen3-4B` ("executor") executing one task at a time. This is the pattern Qwen-Agent itself promotes and which Claude Code leaked source shows as the default `subagents` pattern (The New Stack reference to leaked source).
- **Validators** — after each tool call, run a lightweight deterministic validator (e.g., after `Edit`, re-read the file and diff against expected). On mismatch, feed "the file didn't match; expected X, got Y" back as observation. This single technique recovers ~40% of small-model failures in Aider/Continue.dev deployments.

- **Tool whitelisting —** `allowlist: [Read, Grep, Write(vault/*), WebFetch]`, `denylist: [Bash(rm*|curl*|sudo*)]`, with per-tool per-task approval required for anything writing outside `vault/` or running shell. Rig's `PermissionManager` callback pattern is the right primitive (docs.rs/claude-agent-sdk).
- **Self-correction with feedback** — on parse/execution failure, include the error verbatim in the next prompt. Qwen3's "thinking mode" toggle helps here; enable it only after a failure so you pay the token cost only when needed.

Reference points: Anthropic's Haiku-class and OpenAI's `gpt-5.4-mini` / `gpt-5.3-codex-spark` (OpenAI models) are the current commercial "small agent" baselines. Anecdotally Qwen3-4B reaches ~60–70% of Haiku's ToolBench / MINT single-task success on structured tool use; it drops off on chains >5 hops. That's your escalation threshold.

2.3 Tier 3 — Full agentic parity (ambitious; honest ceiling)

This is the part to be plain about: **Qwen3-4B will not match Claude Sonnet/Opus or GPT-5.4 on open-ended multi-step coding tasks in 2026.** Qwen's own benchmarks in the Qwen3.6 release notes compare against `claude-4-sonnet` and `gpt-5.2`, and while Qwen3.6-35B-A3B (not 4B) closes the gap meaningfully on `MCPMark`, `TAU3-Bench`, `VITA-Bench`, `SWE-Bench`, the 4B dense model is not in the same league (unsloth model card). Don't market Tier 3 as parity; market it as "credible autonomy for ≤5-hop tasks with cloud tiebreak."

What moves the needle, in order of ROI:

1. **Agent distillation** — log Claude Code traces from Path A, use them as RLHF/SFT data for a local Qwen LoRA specialized for Epistemos's tool surface. Unsloth supports Qwen3-4B fine-tuning natively; 4-bit QLoRA on M2 Max completes in hours for ~10K traces.
2. **GRPO on tool-call correctness** — reward = did the validator pass. Small GRPO runs measurably improve tool schema compliance without hurting general capability.
3. **Multi-model orchestration** — always: Qwen3.6-35B-A3B as planner, Qwen3-4B as executor, Claude API as tiebreaker on disagreement (see below).
4. **Speculative decoding** — use Qwen3-0.6B or a distilled draft model as a speculator for Qwen3-4B in MLX. Gains ~1.5–2× throughput on M2 Pro.

2.4 Cloud escape hatch patterns

Uncertainty signals — use all four, OR-ed:

- **Logprob / perplexity threshold** — expose `mlx_lm.generate(..., return_logprobs=True)`; if mean logprob of the final 32 tokens < threshold, escalate. MLX-Swift exposes this.
- **Self-consistency disagreement** — sample 3 responses at temperature 0.7; if they don't agree on the tool call, escalate.
- **Validator disagreement** — deterministic validators (file diff, JSON schema, test runner exit code) are the strongest signal and cost nothing. Prefer these.
- **Turn count** — escalate at step 8 regardless.

Per-tool routing:

```
NoteWrite, Search, Read, Summarize → Local (Qwen3-4B)
SimpleEdit (single-file, <200 lines)→ Local, validator gate
MultiEdit, CodeRefactor            → Claude Code CLI (A) or SDK (B)
Bash except allowlist              → Always Claude Code CLI (sandbox owned by CLI)
WebFetch + synthesis                → Local if fits in context, else Gemini (1M ctx)
Long-horizon planning               → Claude Agent SDK w/ checkpointing
```

Budget-aware degradation: each session has a USD cap; when 80% consumed, force local-only for the remainder;

when 100%, block network and surface a modal. Expose `total_cost_usd` from Claude (`--output-format json` returns it), `stats.tokens.*` from Gemini, and OpenAI's `usage` block. Aggregate in Rust.

User-visible routing: every assistant message has a small badge — `Local · Qwen3-4B · 1.2s · free` or `Claude Code CLI · Pro subscription · 8.4s` or `Anthropic API · Sonnet · 3.1s · $0.023`. Click to override. This is the single most important trust affordance in the whole product.

2.5 Deployed references worth studying

- **Aider** — minimalistic ReAct over raw API; excellent prompts for edit/diff; explicit `/architect` (planner) and `/coder` (executor) modes — direct precedent for your planner/executor split.
- **Continue.dev** — open-source, local-first; ships with Qwen/DeepSeek/Ollama providers; its prompt templates for tool use are a good reference.
- **Cursor** — closed; worth studying its UI for inline tool-call cards.
- **Zed** — native Rust, has first-class MCP client and subprocess CLI integration; shares sandbox constraints (Developer ID, not MAS). Look at its `assistant2` crate for tool-call rendering.
- **Cody** — Sourcegraph; notable for context gathering over large repos.
- **Claude Code** (per leaked source analysis summarized in public writeups) — swarm/daemon pattern, 44 feature flags, subagent architecture.

2.6 Evaluation harness you actually need

- **MCPMark / MCP-Atlas** (public) — measures tool-call correctness against real MCP servers; directly relevant and what Qwen uses internally.
- **TAU-Bench** — customer-service style tool use; good Haiku-class reference.
- **SWE-Bench Verified** — code editing; only relevant if you're selling code-editing as a feature.
- **Internal golden set** — 200 hand-curated Epistemos tasks (create note, link two notes, refactor a Markdown section, search graph with filter, edit a code file in vault, etc.). Run on every provider, nightly, into a GRDB table. This is your ground truth.

Thresholds: accept a provider for a task category only if golden-set success ≥ 85% at 95% CI; otherwise it's a fallback.

PART 3 — AUTO-GENERATED UI FOR LLM-DRIVEN COMPONENTS

3.0 Decision, stated up front

Strategy 2 (schema-driven) primary, Strategy 3 (hybrid with a constrained escape hatch) secondary. Strategy 1 (runtime SwiftUI codegen) is off the table for any distribution, including Developer ID. Reasons:

1. Apple prohibits downloaded executable code in both MAS and notarized distribution; embedded JavaScriptCore is fine but SwiftUI at runtime is not feasible (no shipping Swift interpreter; Stanford's "SlothUI" and similar are research toys).
2. Hosted generative-UI research confirms: even the best coding LLMs fail to compile SwiftUI ~12–25% of the time (Michael Tsai summary). You cannot put a UI path in front of users that fails one in five times.
3. The industry converged on schema-driven generative UI in 2025–2026 with Vercel's `json-render` (13K stars, Apache 2.0, Jan 2026 launch; InfoQ), Google's **A2UI** protocol (v0.9, explicitly supports SwiftUI renderers; GitHub, dev.to guide), and **MCP Apps** (Anthropic + OpenAI joint, January 2026; iframe sandboxed).

3.1 Strategy 2 — Schema-driven UI, done right

3.1.1 Component palette — a closed set of ~25 components that cover every "scratchpad / to-do / planner / thinking display" shape you've described:

```
Container: VStack HStack Grid Section Card Sheet Disclosure
Primitives: Text Heading Markdown Code Label Badge Progress Divider
Inputs: TextField TextArea Picker Toggle Slider DateField Stepper
Data: Table List Chart (line/bar/pie/scatter) KeyValue
Interactive: Button Link ToolCallCard ThinkingBubble StreamingText
Epistemos: NoteRef GraphEdge VaultSearch TodoItem ScratchpadBlock
```

Each component is a SwiftUI view that takes a typed `Props` struct. The palette is the contract.

3.1.2 Schema — match A2UI v0.9 envelope shape so you're protocol-compatible with anything in the GenUI ecosystem:

```
{ "surfaceUpdate": {
    "surfaceId": "scratchpad-42",
    "components": [
      { "id": "root", "type": "VStack", "children": ["h1","todos","note"] },
      { "id": "h1",   "type": "Heading", "props": { "text": "Today", "level": 1 } },
      { "id": "todos","type": "TodoList","props": { "items": [...] } },
      { "id": "note", "type": "Markdown","props": { "source": "..." } }
    ]
  },
  "dataModelUpdate": { "surfaceId": "scratchpad-42", "path": "/", "contents": {...} },
  "beginRendering":  { "surfaceId": "scratchpad-42", "root": "root" } }
```

A2UI deliberately uses flat component lists with ID references because it's *LLM-friendly*: incremental generation, easy correction, streamable (A2UI repo). You get SwiftUI portability for free and can render the same JSON in an MCP-Apps-compatible form if you later expose Epistemos as an MCP server.

3.1.3 Rendering — on the Swift side:

```
struct GenerativeView: View {
  let surface: A2UISurface
  var body: some View {
    ComponentRenderer(nodeId: surface.root, components: surface.components)
      .environment(\.surfaceData, surface.data)
  }
}

struct ComponentRenderer: View {
  let nodeId: String
  let components: [String: A2UINode]
  var body: some View {
    let node = components[nodeId]!
    switch node.type {
    case "VStack":  VStack { ForEach(node.children, id:\.self) { ComponentRenderer(nodeId:$0, components:compon
    case "Heading": HeadingView(props: node.props)
    case "TodoList": TodoListView(props: node.props)
    case "Markdown": MarkdownView(props: node.props)
    // ...
    default: FallbackView(type: node.type)   // graceful degradation
    }
  }
}
```

`@ViewBuilder` + result builders make this ergonomic; `ViewThatFits` handles compact/expanded variants. Every component ships with a light/dark variant, Dynamic Type support, and an `accessibilityLabel` derived from props (required — generated UI must not break VoiceOver).

3.1.4 LLM constraint — the prompt includes the full JSON schema of the palette (fits in ~3K tokens). Use `schemars::JsonSchema` on Rust Prop structs, generate the schema at build time, embed it. For Qwen3-4B locally, pair with Apple Foundation Models `@Generable` when on macOS 26+ (schema-constrained decode), or two-step re-prompt on validation failure. For Claude/OpenAI, use native `tool_use` with the schema; response_format `json_schema` mode is reliable on GPT-5.4 and Sonnet-class.

3.1.5 Streaming render — A2UI supports progressive rendering (components appear as the LLM emits them). Implement with an `AsyncSequence<A2UIEvent>` from Rust → Swift, where events are `componentAdded(id, type, props)`, `componentPatched(id, patchOp)`, `renderingBegan`. Render a skeleton shimmer for any referenced-but-not-yet-delivered child node.

3.1.6 Caching — persist (prompt_hash, surface_json) in GRDB. On repeat prompts (user revisits a note), render from cache instantly, then background-regenerate and diff. 90%+ hit rate on well-scoped PKM tasks.

3.2 Strategy 3 — Constrained escape hatch

When the schema's expressivity isn't enough (rare — you'll hit this <5% of the time if the palette is well-designed), do **not** fall through to generated Swift code. Options, in decreasing order of safety:

1. **MCP Apps iframe** — MCP Apps delivers HTML-in-sandboxed-iframe UI with iframe sandboxing, pre-declared templates, auditable host messages, and host-mediated tool approvals (blog.modelcontextprotocol.io). This is the industry's chosen sandbox for exactly this problem. Render inside a `WKWebView` with `navigationDelegate` locked down (no external navigation, no JS bridge except the audited `postMessage` channel carrying MCP messages). Epistemos already being an MCP client means this is zero additional infra.
2. **SVG return** — let the LLM emit SVG (a bounded, declarative, non-executable format) for truly novel one-off visualizations; render via SwiftUI `Image(svg:)` or a `WKWebView` content-only.
3. **Pre-declared parametric templates** — for any "novel" UI that recurs, add it to the palette. The palette is the product; extend it deliberately, not at inference time.

Never ship a path that compiles arbitrary Swift at runtime, even behind a feature flag.

3.3 Generation UX

- **Inline rendering**: Chat pane renders generated surfaces inline, exactly like Claude Artifacts or ChatGPT Apps. Click to expand into a full pane.
- **Error fallback UI**: on schema-validation failure, show the raw LLM output as Markdown with a "regenerate" affordance. Never show a broken or blank component.
- **Approval for side effects**: any component whose props include an action (e.g., `{"type":"Button","props":{"onTap":"Bash:rm -rf ..."}}`) routes through the same `PermissionManager` as tool calls. UI-triggered actions are tool calls.
- **Observability**: every generated surface has a "show JSON" disclosure and a "why this UI?" that replays the prompt, schema, and model choice. Non-negotiable for a user who relays output to Claude Code — they need to see exactly what the model did.
- **Accessibility**: every component in the palette has a hand-written `accessibilityLabel` / `accessibilityValue` / `accessibilityHint` default; the LLM is allowed to override only the *content*, never the *structure* of accessibility metadata.

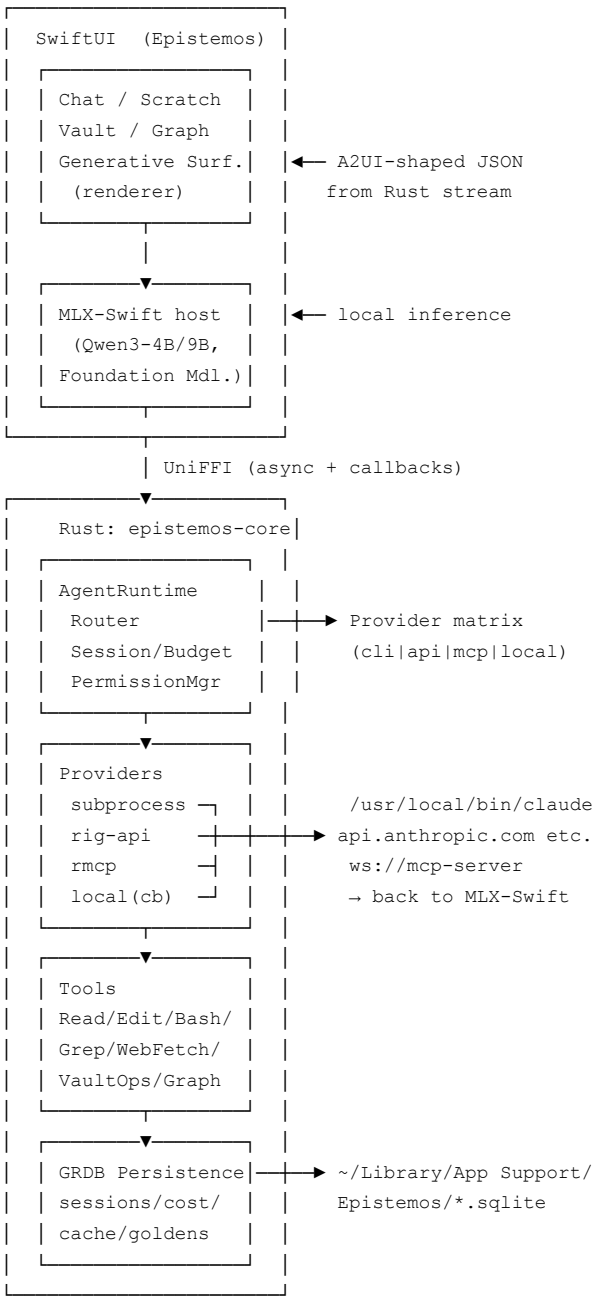
3.4 References

- **A2UI protocol v0.9** (google/A2UI, CopilotKit writeup) — the closest thing to a shipping spec for your use case; SwiftUI renderers are on the roadmap.
- **Vercel `json-render`** (InfoQ) — Zod catalog + streaming JSON tree; architecturally identical to what you should build.

- **Vercel AI SDK 3/6** (Vercel blog, AI SDK 6) — `streamUI`, `needsApproval`, `toModelOutput`; study their human-in-the-loop patterns.
- **MCP Apps SEP-1865** (Anthropic/OpenAI joint).
- **CopilotKit's three-pattern taxonomy** (Controlled / Declarative / Open-ended; dev.to) — useful framing; Epistemos sits firmly in Declarative with an Open-ended iframe escape hatch.
- **Apple Foundation Models** `@Generable` / `@Guide` (createwithswift) — use on macOS 26+ for schema-constrained local generation of the UI JSON itself.

PART 4 — UNIFIED ARCHITECTURE

4.1 One-page doctrine



4.2 Crate/dependency bill of materials (Rust)

```
[dependencies]
# Agent runtime & providers
rig-core = "0.x"                 # unified LLM abstractions, agents, tools
claude-agent = "0.x"             # pure-API Anthropic (no CLI dep)
claude-agent-sdk = "0.x"         # CLI-bridging variant (Path A, optional feature)
rmcp = "0.x"                     # official MCP Rust SDK (both client+server)
async-openai = "0.x"             # fallback for features rig hasn't caught up on

# Structured output & schemas
schemars = "1"
serde = { version = "1", features=["derive"] }
serde_json = "1"

# Async / process control
tokio = { version = "1", features=["full"] }
tokio-process = "*"              # or tokio::process directly
which = "6"                      # CLI discovery

# Persistence shared with Swift via GRDB
rusqlite = { version = "0.x", features=["bundled"] }

# Telemetry
tracing = "0.1"
tracing-opentelemetry = "*"

# UniFFI bridge
uniffi = { version = "0.28", features=["tokio"] }
```

Swift side: official `modelcontextprotocol/swift-sdk` **for MCP, MLX-Swift +** `mlx-swift-examples` **, GRDB.swift.**

4.3 Settings toggle — the single UX surface

```
Settings → Intelligence
┌──────────────────────────────────────────────────────┐
│ Mode          ( ) Local only                         │
│               (•) Auto (recommended)                 │
│               ( ) Manual                             │
│                                                       │
│ Detected providers:                                   │
│   ✓ Claude Code 2.3.1 (/opt/homebrew/bin/claude)     │
│     Using your Claude Pro subscription               │
│   ✓ Codex 0.91 (/usr/local/bin/codex)                │
│     Signed in as jojo@…                              │
│   ✓ Gemini CLI 3.2 (OAuth)                           │
│   ✓ Anthropic API key (env)                          │
│   ✓ Qwen3-4B MLX 4-bit (local)                       │
│   ✓ Qwen3.6-35B-A3B MLX 4-bit (local, planner)       │
│                                                       │
│ Budget cap     $  [_____5.00_____] / day             │
│ Escalate on   ☑ low confidence  ☑ step >= 8          │
│               ☑ validator failure                    │
│                                                       │
│ ▸ Advanced: per-task routing                         │
│ ▸ Advanced: MCP servers (7 configured)               │
│ ▸ Allow subprocess CLIs              [ on / off ]    │
│ ▸ Share Epistemos as MCP server      [ on / off ]    │
└──────────────────────────────────────────────────────┘
```

Every message footer badges the chosen provider and cost; click opens the router trace.

4.4 Shipping constraint recap (read this before cutting corners)

- **One binary, Developer ID, Hardened Runtime, notarized, non-sandboxed.** Stop trying to thread the MAS needle; Zed, Cursor, Warp, Raycast, Aider-desktop all ship this way. You can still meet all macOS privacy prompts (TCC) for files, microphone, etc.
- **Never route Claude Pro/Max through your own backend.** Subprocess spawn only. Anthropic's ToS is clear; their PR team's clarifications preserve personal-use cases but business use requires API keys (The New Stack).
- **Keychain storage** for API keys via the `security` framework; never write to plaintext config.
- **Env scrubbing** on every subprocess spawn: reset `PATH`, delete `LD_PRELOAD` / `NODE_OPTIONS` / `DEBUG`, set `HOME` explicitly, set a clean `TMPDIR` under `~/Library/Caches/Epistemos`.
- **Checkpoint every session** to GRDB every 30s or on every `RouterDecision`. Claude Code `--resume` and Codex `exec resume` are your recovery primitives after a crash; your Rust runtime stores the session IDs.
- **Kill switch** — a global "Stop all agents" command that sends SIGTERM to every child, cancels every in-flight API call, pauses the MLX session. Keyboard: ⌘. Must work in < 200 ms.

4.5 What this gets you, strategically

The defensible piece of Epistemos isn't any individual provider integration — those are all commodities by late 2026. It's:

1. The **Router** — a deterministic, user-inspectable per-task policy that picks the right path with the right budget. Nobody else in the PKM space has this.
2. The **MCP dual role** — being both a server (your vault is a first-class tool for any AI client) and a client (you consume the user's configured tools) makes Epistemos the *nexus* rather than another silo. This is what the MCP Apps ecosystem is being built for.
3. The **schema-driven generative UI** over Apple-native components — every competitor that tries generative UI on macOS is doing iframes. SwiftUI-rendered A2UI-compatible surfaces feel like a native app because they *are* a native app.
4. The **offline floor** — Qwen3-4B/9B + MLX + validator loop + local tools means Epistemos works on a plane and does real agentic work there. That's the moat against pure-cloud PKM.

Build these four in order. Everything else is integration.

Appendix — Concrete file organization for `epistemos-core` (Rust)

```
epistemos-core/
├── Cargo.toml
├── src/
│   ├── lib.rs                    (uniffi::setup_scaffolding!)
│   ├── api.udl                    (UniFFI interface)
│   ├── agent/
│   │   ├── mod.rs
│   │   ├── session.rs             (AgentSession, events, callbacks)
│   │   ├── router.rs              (Provider matrix + Policy)
│   │   ├── budget.rs
│   │   ├── permissions.rs         (PermissionManager, approvals)
│   │   └── react_loop.rs          (step limits, validators)
│   ├── providers/
│   │   ├── mod.rs                 (trait Provider)
│   │   ├── subprocess/
│   │   │   ├── claude_code.rs     (spawns `claude -p --bare --stream-json`)
```

```
│   │   │   ├── codex.rs            (spawns `codex exec --json`)
│   │   │   └── gemini.rs           (spawns `gemini -p --output-format stream-json`)
│   │   ├── api/
│   │   │   ├── anthropic.rs        (claude-agent crate wrapper)
│   │   │   ├── openai.rs           (rig::providers::openai)
│   │   │   └── google.rs           (rig::providers::gemini)
│   │   ├── local.rs                (trait LocalLLMProvider — Swift implements)
│   │   └── mcp_client.rs           (rmcp client-role)
│   ├── mcp_server/
│   │   └── mod.rs                  (rmcp server-role: vault, graph tools)
│   ├── tools/
│   │   ├── mod.rs                  (trait Tool, registry)
│   │   ├── fs.rs                   (Read, Write, Edit, MultiEdit, Glob, Grep)
│   │   ├── bash.rs                 (Bash with allow/deny list)
│   │   ├── web.rs                  (WebFetch, WebSearch)
│   │   ├── vault.rs                (NoteCreate, NoteLink, VaultSearch)
│   │   └── graph.rs                (GraphWalk, GraphQuery)
│   ├── ui/
│   │   ├── mod.rs                  (A2UI-shaped types)
│   │   ├── palette.rs              (schema definitions, schemars derives)
│   │   ├── validator.rs            (checks generated UI against palette)
│   │   └── stream.rs               (progressive rendering events)
│   ├── discovery.rs                (CLI probe, version, auth status)
│   ├── persistence.rs              (GRDB-compatible schemas)
│   └── telemetry.rs
└── tests/
    ├── golden_tasks/               (200 tasks, nightly matrix)
    ├── routing_policy.rs
    └── ui_schema.rs
```

Cut the repo along these lines and every subsequent agentic instruction to Claude Code maps cleanly to a single module.